CLSU Feedback Management System Architecture

System architecture documentation

Version 1.0 | 19 October 2026 | Proposed implementation baseline

Purpose and architectural decision

This document defines the proposed architecture of the CLSU Feedback Management System and Service Tracker. It guides the capstone development team and MISO reviewers in implementing service requests, feedback collection, tracking, reporting, and controlled AI agent actions.

The system will use one React application, one modular Laravel backend, and a centralized MySQL database. Both the web API and Model Context Protocol tools will invoke shared application services. This arrangement keeps business rules consistent and supports a manageable deployment for the development team.

System scope

The first release covers public general feedback, authenticated service requests, MISO review and assignment, personnel progress updates, clarification, completed-service evaluations, notifications, and standard reports. An MCP connection will let authorized administrators ask an AI agent what needs attention and instruct it to perform defined operational updates.

Layer	Selected approach
Frontend	React with TypeScript and Vite; responsive role-based interfaces
Application	Laravel API, policies, validation, domain services and MCP server
Persistence	MySQL and private attachment and report storage
Background work	Laravel queue worker and scheduler
AI integration	Compatible external MCP agent; server-side runtime for embedded chat

Design basis

The architecture follows the service-job and feedback requirements in the ITRM 3100 research draft and the requester, mobile, administrative, and tentative assistant screens in the FEEDBACK MANAGEMENT Figma file. The first-release MCP boundaries reflect the agreed capability scope. The architecture describes intended behavior and does not claim that the system has already been implemented or tested.

1 Architecture overview

The application uses a modular monolith: Laravel modules share a deployment and database while keeping their responsibilities separate. React communicates with the application API over HTTPS. External AI agents connect to a protected MCP endpoint; embedded chat uses a server-side agent runtime.

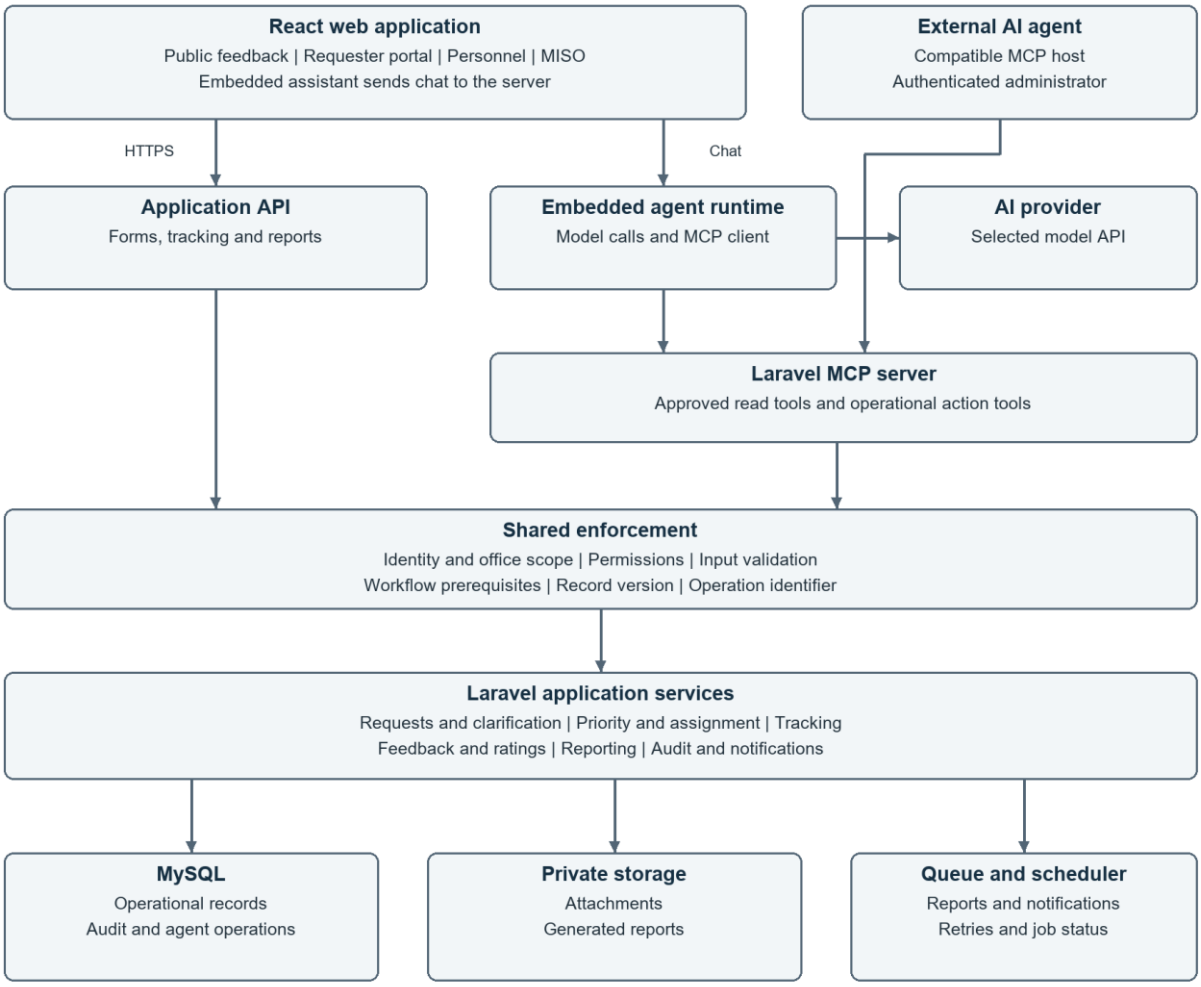


Figure 1 Proposed logical system architecture

Common execution path

API controllers and MCP tools call the same application services. Every protected operation resolves the caller identity, checks permissions and office scope, validates inputs, and applies workflow rules. Public feedback uses a separately restricted submission route. Operational changes and their audit entries are committed together where possible.

Agent and frontend separation

The model selects among exposed tools and explains returned results. Laravel determines whether an action is allowed and persists successful changes. The agent receives a structured result, while React reloads the affected record or responds to an application event. Model credentials remain on the server or in the external agent host.

2 Application modules and user access

Module	Responsibility
Identity and access	Accounts, roles, office memberships and policy checks
Requests and clarification	Unique job numbers, service details, revisions and clarification history
Priority and assignment	Urgency and impact assessment, official priority, eligible staff and assignments
Tracking	Permitted transitions, actions taken, timestamps and note visibility
Feedback and ratings	General feedback and one requester evaluation per completed job
Reporting	Consistent aggregates, period filters and private report exports
Notifications and audit	Application notices and traceable changes from web and MCP actions

Access model

Actor	Allowed access
Public visitor	Submit general feedback; no private request lookup by job number
Authenticated requester	Create requests; view own records; clarify and rate eligible completed jobs
Service personnel	View assigned work and perform permitted progress updates
MISO administrator	Review and route requests within authorized scope; assign, report and use MCP
Connected AI agent	Use approved tools within the connected administrator permissions and tool scopes

Office and role boundaries

The requesting office and handling office are separate fields. Office membership does not automatically grant access to all university records. Student, faculty, staff, visitor, and alumni classifications describe a user; operational roles determine permissions. The first release exposes operational MCP tools to authorized MISO administrators.

Visibility of service records

Requester-visible updates and internal operational notes are explicitly distinguished. Internal notes are excluded from requester responses. Private attachments require record-level authorization even when a user knows the file identifier or job number.

3 Service workflows and business rules

Request submission and review

An authenticated requester submits the service type, subject, location, requesting affiliation, concern, urgency, and impact. Laravel validates the submission and generates a unique job number. MISO verifies the details, confirms the handling office and category, calculates priority using configured rules, and assigns eligible personnel.

Clarification and progress

MISO can request missing details and place the request in For Clarification. The requester resubmits under the same job number; revisions and messages remain in the history. Personnel record work and permitted status changes. On Hold requires a reason; resuming follows the configured transition rules.

Status	Meaning or prerequisite
Pending	Received and awaiting review or assignment
For Clarification	A recorded clarification request needs a requester response
Assigned	Eligible personnel are assigned to the request
In Progress	Work has started and an operational update is recorded
On Hold	Work is paused with a recorded reason
Completed	Authorized staff record actions taken and the service outcome

Completion and evaluation

Authorized human staff mark work Completed after documenting the outcome. The first-release agent cannot complete a service. The requester may submit one optional evaluation linked to the job and personnel; not receiving a rating does not undo completion.

General feedback

General feedback can be submitted independently of a service job. Optional contact details allow follow-up. A supplied job number is an unverified reference until ownership or a secure invitation is verified; it grants neither record access nor permission to submit the verified job evaluation.

Priority and measurement

Requester-reported urgency and impact are retained separately from the official priority. Laravel applies the MISO-approved matrix; authorized exceptions require a reason. Reports distinguish first response time from completion time, define the reporting period and denominator, and show the rating count beside satisfaction averages. No response targets are assumed until MISO configures them.

4 Data architecture and record integrity

Entity group	Main records and relationships
Access and organization	Users, offices, memberships, roles and permissions
Service configuration	Service types, categories and versioned priority rules
Service requests	Requester, requesting office, handling office, job number, details, official priority and status
Operational history	Multiple assignments, status transitions, clarification messages, revisions and progress notes per request
Feedback	Independent general feedback; zero or one verified requester evaluation per completed job
Supporting records	Attachment metadata, notifications, report jobs, agent operations and append-only audit entries

Database constraints

Foreign keys preserve valid relationships. Unique constraints enforce job-number uniqueness, one verified evaluation per job, and operation-identifier uniqueness within the appropriate caller and tool scope. Job-number generation uses transactional sequencing or an equivalent concurrency-safe strategy rather than counting existing rows.

Atomic updates and concurrent editing

Each operational write checks an expected record version. If another user has changed the record, Laravel rejects the stale action and returns the current version for review. Related state changes and audit entries are saved in a database transaction. An assignment also checks personnel eligibility and current request state.

Retry and notification behavior

Every action tool accepts a unique operation identifier. Repeating the same operation returns its recorded result; reusing an identifier with different parameters is rejected. Notifications and exports are dispatched after commit. Workers also prevent duplicate delivery where feasible, since a database transaction cannot guarantee exactly-once delivery to an external email service.

Storage and lifecycle

MySQL stores attachment metadata; file content and generated reports remain in private storage. Downloads use authenticated authorization checks or short-lived authorized links. The implementation must define retention, backup frequency, restoration ownership, and deletion procedures with MISO before production use. The MCP tool set does not expose deletion.

5 MCP tools and agent interaction

MCP exposes specific business capabilities to compatible AI hosts. It supplies a tool interface; the connected host runs the agent and model interaction. The embedded assistant additionally requires a server-side runtime and configured provider. The initial tool catalog is shown below.

Tool	Purpose and limit
search_requests	Filter and paginate authorized requests; no arbitrary SQL
get_request_details	Return permitted details and timeline with record version
get_attention_queue	Return rule-based ranking, evidence and configured deadlines
get_personnel_workload	Return recorded eligibility, availability and active assignments
get_feedback_metrics	Return backend-calculated aggregates, periods and sample counts
get_operation_result	Retrieve the caller-authorized outcome after a timeout or retry
assign_request	Assign or reassign eligible personnel on explicit instruction
set_request_priority	Apply rule-based priority or authorized documented exception
add_internal_note	Append an explicitly requested operational note
request_clarification	Record specified message and permitted status change
transition_request_status	Apply configured operational transitions; exclude Completed
request_report and get_report_job	Create a standard export and check its final outcome

Recommendation followed by action

For a question such as Which requests should we work on first, the agent reads the attention queue and recorded workload, then explains its recommendation with supporting job references. Reading and recommending do not authorize updates. An instruction such as Assign this request to Mark authorizes that defined action, subject to server checks.

Tool contract

Reads use bounded filters, pagination and minimal fields. Writes include the target identifier, validated parameters, expected version, operation identifier and reason. The server derives identity from authentication. Results contain the operation identifier, affected record, resulting version and success or error state. An accepted report job is not reported as a completed export.

Permission denial, invalid workflow, stale version, invalid input and temporary service failure return distinct errors. The agent reports actual outcomes and does not claim success after a failed or merely queued operation.

6 Security boundaries and agent limitations

Authentication and authorization

Protect the remote MCP endpoint with HTTPS and a supported authentication method. A controlled pilot can use revocable, scoped user tokens; external multi-user connections should use a compatible OAuth flow. Check roles, tool scopes, office scope and target-record permission on every call. A model-supplied user identifier cannot establish authority.

Authorization of actions

Explicit administrator instructions authorize the specified operational update. Where a separate confirmation is required, Laravel verifies an approval record bound to the user, exact target, parameters, record version and expiry. Approval is consumed once. Host approval prompts and tool annotations do not replace server authorization. Instructions embedded in service descriptions or feedback cannot authorize actions.

First release exclusions

- No arbitrary SQL, shell execution, source-code changes, or unrestricted database access.
- No account, role, permission, or system-configuration administration through MCP.
- No deletion or alteration of original requester statements, client ratings, or audit history.
- No agent-driven completion, fabricated service outcomes, or unattended bulk reassignment.
- No arbitrary outbound email, unrestricted exports, or continuous autonomous monitoring.

Privacy and untrusted content

Return only fields needed for the task and exclude passwords, tokens and unnecessary contact information. Validate attachments and public input, apply submission and tool rate limits, and separate private notes from public updates. Treat all submitted content and retrieved attachments as untrusted data. MISO must approve provider data handling and retention settings before personal data is processed by an external model.

What AI recommendations can establish

Recommendations use recorded urgency, impact, request age, configured deadlines and workload. Negative sentiment alone does not establish urgency. Recorded availability does not prove that a technician is physically available. The agent can explain trends and options, but cannot guarantee diagnosis accuracy or replace staff verification.

7 Deployment and failure handling

Initial deployment topology

Deploy the React production build and Laravel API under one HTTPS domain where practical. Laravel connects to MySQL and private file storage; a supervised queue worker handles exports and notifications, and a scheduler handles defined maintenance tasks. Start with a database-backed queue if suitable for measured load; Redis is optional rather than a first-release requirement.

A private campus deployment requires network access from the chosen agent host. An external MCP client cannot reach a campus-only endpoint without an approved network path. The AI provider also requires outbound access when used. Package versions, PHP extensions and client transport compatibility must be verified before deployment.

Frontend synchronization

After a successful mutation, React refetches the affected data. Polling can refresh other open views in the initial release; broadcast events may be added when needed. Persisting a database change does not automatically refresh a browser. A visible result should distinguish successful changes from queued, failed or conflicted operations.

Failures and recovery

Condition	Required behavior
Model or MCP unavailable	Keep ordinary request, assignment, tracking and feedback functions available
Tool timeout after submission	Retrieve the operation result using its identifier before retrying
Concurrent record change	Reject stale write; refresh details and re-evaluate the action
Report still running	Return a job identifier and status; provide download only after success
Worker or email failure	Record failure, apply bounded retries and make operational failure visible

Operational controls

Set connection and request timeouts, worker supervision, retry limits and job monitoring. Store provider credentials and signing secrets outside the frontend and repository. Log failures without exposing sensitive request content or credentials. Test database and private-file restoration together.

Capacity and availability

The architecture does not assume unlimited throughput or continuous agent execution. Rate limits, model costs, context limits, host timeouts and university network conditions constrain use. Load testing will establish practical capacity; the research draft server specification is a planning baseline rather than a validated capacity claim.

8 Verification and implementation decisions

Acceptance criteria

Area	Required evidence before release
Request lifecycle	A requester submits, clarifies, tracks and optionally evaluates a completed job
Permissions	Cross-user and cross-office access attempts fail through both API and MCP
Priority and reports	Configured matrix, time calculations, denominators and rating counts produce expected results
Agent actions	Authorized assignment and progress changes persist; excluded operations are unavailable
Integrity	Stale versions fail; repeated operation identifiers do not duplicate writes
Privacy	Internal notes and attachments remain restricted; injected instructions cannot grant access
Resilience	Provider failure leaves core workflows usable; report jobs and failed notifications remain traceable

Decisions to confirm during implementation

MISO must confirm public feedback eligibility, guest access and moderation; the urgency and impact matrix and response targets; handling-office responsibilities; allowed status transitions; report period and business-hour definitions; provider data handling; retention and backup policies; and production hosting. These settings must be finalized before they become enforced business rules.

The research manuscript must also align its scope with the inclusion of third-party AI integration and distinguish MCP from the language model. Exact Laravel, PHP, MCP and AI runtime versions will be pinned after compatibility checks.

References

Research draft: Development and Enhancement of Web Based Feedback Management System and Tracker for CLSU Services, ITRM 3100.md. Local reference supplied by the project team.

Design reference: FEEDBACK MANAGEMENT, Figma. https://www.figma.com/design/PeQN6nVBQbPNXWoTU32Mhm/FEEDBACK-MANAGEMENT?node-id=0-1

Laravel MCP documentation. https://laravel.com/framework/docs/13.x/mcp

Laravel AI SDK documentation. https://laravel.com/framework/docs/13.x/ai-sdk

Laravel authorization documentation. https://laravel.com/framework/docs/13.x/authorization

Laravel queues documentation. https://laravel.com/framework/docs/13.x/queues

Documentation references reviewed 19 October 2026. Framework features are distinguished from the application rules and controls specified in this document.